

SplitEase

Smart Shared Expense Management System for Students & Roommates

A comprehensive software design document detailing core architectural principles, high-level structural styles, greedy algorithmic debt simplification, and user experience engineering.

Project Title:	SplitEase (Smart Shared Expense Management System)
Document Version:	1.0.0 (Release Architecture Baseline)
Author / Lead:	ABHIJEET AND SHRISH & Engineering Team
Target Environment:	Node.js 20, Next.js 14 App Router, Express, MongoDB 7, Docker
Primary Stakeholders:	College Students, Flat Residents, Hostel Roommates
Date:	September 2026

Executive Summary

Managing shared household expenditures (e.g., apartment rent, groceries, electricity, cooking supplies, high-speed WiFi) in student accommodations has historically been hindered by fragmented WhatsApp messages, mental approximations, and manual spreadsheets. These informal systems result in lost expenditures, miscalculated balances, and uncomfortable interpersonal friction among roommates.

SplitEase provides an end-to-end web platform engineered specifically for students. It establishes a verifiable, single source of truth for group expenditures, automates fractional expense splitting (Equal, Custom Amounts, Percentages), continuously maintains personal net ledgers, and minimizes cash settlements through a **Greedy Two-Pointer Debt Simplification Algorithm**. This document specifies the complete software architecture, technical design principles, database model, algorithmic foundations, and user interface design.

1. System Context, Problem Statement & Scope

1.1 Problem Statement

In shared student flats and hostel apartments, daily purchasing happens impulsively. Roommate A buys groceries on Monday; Roommate B pays the monthly internet bill on Wednesday; Roommate C pays electricity on Friday. Tracking these cross-cutting obligations manually produces:

- **Unbalanced Fractional Math:** Splitting uneven bills (e.g. ₹1,249 across 3 flatmates) produces rounding discrepancies that accumulate over months.
- **Circular Debt Cycles:** Without simplification, 4 roommates end up making 6 to 12 fragmented peer-to-peer transfers, confusing who owes whom.
- **Lack of Transparency:** Casual chat logs lack audit trails, dispute prevention, and category-level budget insights.

1.2 Target User Personas & Goals

Persona	Context & Frustrations	Key Engineering Goals
Rahul (Age 20) Flat Resident	Frequently pays for flat groceries and Swiggy food orders. Forgets who repaid him and dislikes manual math.	Record expenses in <10 seconds; instant visibility of who owes him; zero calculation effort.
Priya (Age 21) Group Organizer	Manages a 4-member flat lease; pays master utility and rent bills; requires monthly spending accountability.	Complete spending visibility; organized category analytics; dispute-free debt settlement records.
Admin / Creator Flat Lead	Onboards new roommates, creates flat expense groups, and archives departed tenants.	Role-based group access control; seamless member invitations; persistent ledger integrity.

1.3 Scope & Boundary Conditions

SplitEase is purposely scoped for high-speed student flatmate workflows:

- **In-Scope (MVP & Core):** Secure JWT authentication, group management, flexible expense split calculations, automated ledger maintenance, greedy debt simplification, manual settlement logging, and categorical spending analytics.
- **Out-of-Scope (Off-Platform):** Direct fiat payment gateway integration and banking KYC compliance. Indian students universally utilize external UPI applications (Google Pay, PhonePe, Paytm); forcing in-app payment processing introduces unnecessary transaction fees and compliance barriers.

2. Core Software Design Principles Applied

SplitEase strictly adheres to established object-oriented and modular software engineering principles to ensure maintainability, testability, and clean code separation.

2.1 Abstraction

How Applied: Abstraction hides low-level database operations and transport plumbing behind clear, expressive domain contracts.

- **Data Layer Abstraction:** Mongoose schemas abstract low-level MongoDB BSON documents, providing strict type validation, default timestamps, and embedded subdocuments without exposing driver connection mechanics.
- **Client Network Abstraction:** The frontend Axios instance in `client/lib/api.js` encapsulates base URL routing, JSON serialization, and automatic JWT Bearer token attachment via request interceptors.

Code Snippet 1: Client API Service Abstraction (`client/lib/api.js`)

```
import axios from 'axios';

const api = axios.create({
  baseURL: process.env.NEXT_PUBLIC_API_URL || 'http://localhost:5000/api',
  headers: { 'Content-Type': 'application/json' }
});

// Abstract token injection from UI components
api.interceptors.request.use((config) => {
  const token = typeof window !== 'undefined' ? localStorage.getItem('token') : null;
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

export default api;
```

2.2 Modularity

How Applied: The codebase is decomposed into independent, interchangeable modules organized by feature domain.

- **Backend Separation:** Discrete modules for Routing (`routes/`), Business Logic (`controllers/`), Data Schema (`models/`), and Cross-Cutting Concerns (`middleware/`).
- **Frontend Atomic Components:** Reusable UI building blocks (`Button.js` , `Card.js` , `Input.js` , `Sidebar.js`) decoupled from business logic and page routing.

Code Snippet 2: Modular Route Definition (`server/src/routes/expenseRoutes.js`)

```
const express = require('express');
const router = express.Router();
const { createExpense, getGroupExpenses, deleteExpense } = require('../controllers/expenseController');
const protect = require('../middleware/authMiddleware');

router.use(protect); // Route-level middleware modularity
router.route('/').post(createExpense);
router.route('/group/:groupId').get(getGroupExpenses);
router.route('/:id').delete(deleteExpense);

module.exports = router;
```

2. Core Software Design Principles (Continued)

2.3 High Cohesion

How Applied: Each module and controller has a single, tightly defined responsibility (Single Responsibility Principle):

- `authController.js` : Exclusively responsible for credential hashing, user registration, and JWT token issuance.
- `expenseController.js` : Dedicated strictly to validating expense payloads and calculating split apportionments.
- `balanceController.js` : Focused exclusively on net ledger calculation and executing the debt simplification algorithm.
- `settlementController.js` : Manages debt lifecycle transitions (Pending → Completed).

Code Snippet 3: Cohesive Net Balance Ledger Computation (server/src/controllers/balanceController.js)

```
// Helper to calculate raw balances with high functional cohesion
const calculateRawBalances = async (groupId) => {
  const expenses = await Expense.find({ group: groupId });
  const settlements = await Settlement.find({ group: groupId, status: 'Completed' });
  const balances = {}; // { userId: net_amount }

  // 1. Process Expenses
  expenses.forEach(exp => {
    const paidByStr = exp.paidBy.toString();
    balances[paidByStr] = (balances[paidByStr] || 0) + exp.amount;

    exp.splits.forEach(split => {
      const userStr = split.user.toString();
      balances[userStr] = (balances[userStr] || 0) - split.amount;
    });
  });

  // 2. Process Settlements (Settler +amt, Receiver -amt)
  settlements.forEach(s => {
    balances[s.fromUser.toString()] = (balances[s.fromUser.toString()] || 0) + s.amount;
    balances[s.toUser.toString()] = (balances[s.toUser.toString()] || 0) - s.amount;
  });

  return balances;
};
```

2.4 Low Coupling

How Applied: Components interact through well-defined, minimal interfaces without internal dependency leaking.

- **Client-Server Decoupling:** The Next.js frontend communicates with the Express backend strictly via stateless JSON REST APIs. The frontend has zero knowledge of MongoDB internals, while the backend has zero knowledge of React state.
- **Middleware Authentication Decoupling:** Controllers receive `req.user` without knowing how JWT tokens were unpacked or verified.

Code Snippet 4: Decoupled JWT Authentication Guard (server/src/middleware/authMiddleware.js)

```
const jwt = require('jsonwebtoken');

const protect = async (req, res, next) => {
  let token = req.headers.authorization?.startsWith('Bearer')
    ? req.headers.authorization.split(' ')[1] : null;

  if (!token) return res.status(401).json({ success: false, message: 'Not authorized, no token' });
```

3. High-Level System Architecture

3.1 Architectural Style: 3-Tier Layered Client-Server

SplitEase adopts a **3-Tier Layered Client-Server Architectural Style** orchestrated via Docker Compose.

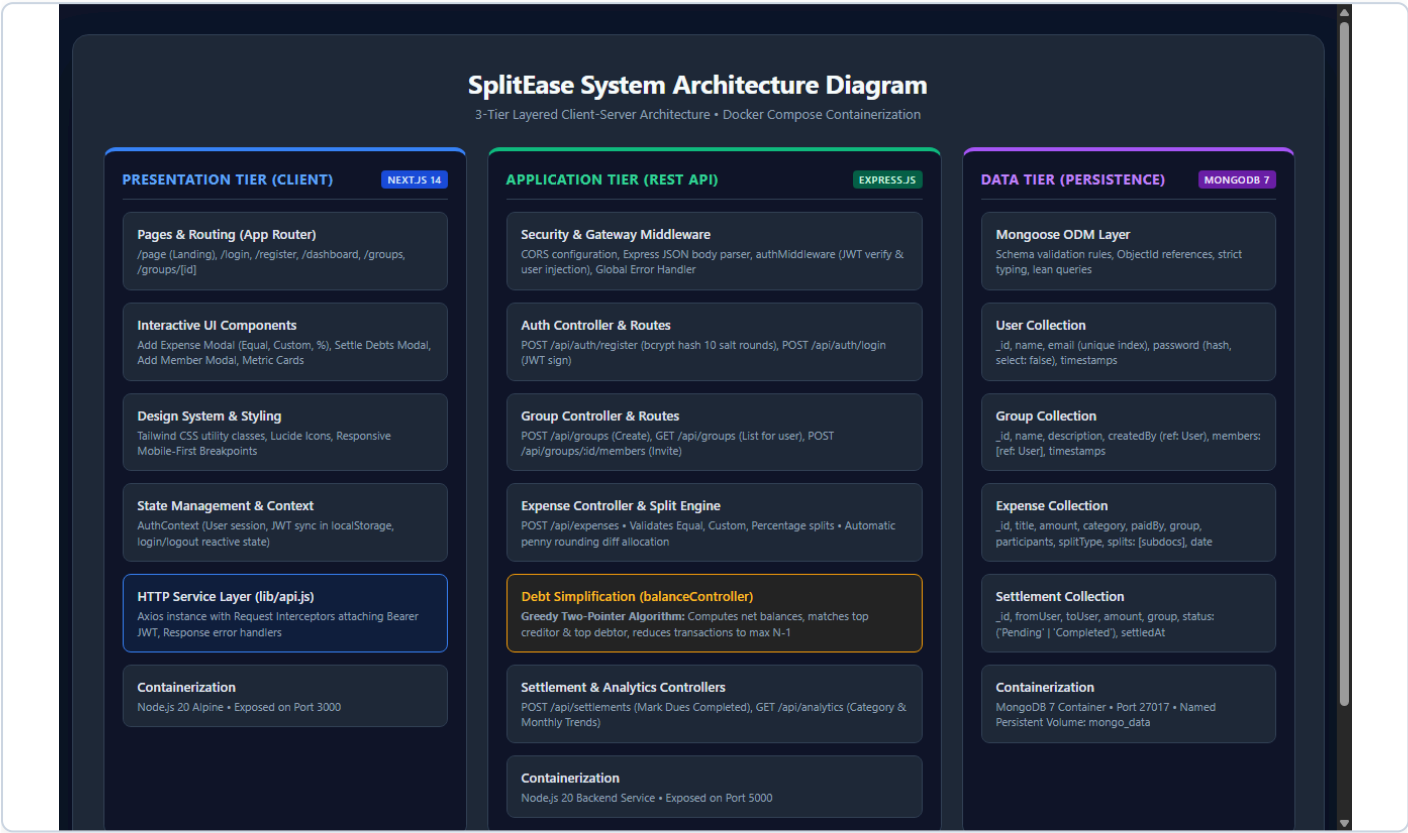


Figure 3.1: SplitEase 3-Tier Architecture Diagram (Rendered from Draw.io source)

3.2 Tier Breakdown & Rationale

- **Tier 1: Presentation Tier (Next.js 14 App Router):** Built with React and Tailwind CSS. Employs client-side state hydration, interactive modals, and Lucide React iconography. Provides responsive mobile/desktop layouts.
- **Tier 2: Application / Business Logic Tier (Node.js + Express REST API):** Implements RESTful endpoints, request validation, authentication guards, and the greedy debt simplification algorithmic pipeline.
- **Tier 3: Persistence Tier (MongoDB 7 + Mongoose ODM):** Document-oriented data store chosen for flexible JSON schema alignment, subdocument nesting for expense splits, and horizontal scalability.

Why this style? The 3-Tier Layered style enforces strict separation of concerns, guarantees independent deployability of frontend and backend services, facilitates local containerized reproduction via Docker Compose, and allows seamless future extensions (e.g. native mobile client) without modifying server-side business logic.

4. Data Architecture & Database Design

4.1 Entity-Relationship Architecture (Mongoose Document Schema)

SplitEase utilizes MongoDB with Mongoose ODM to model four core domain entities: `User` , `Group` , `Expense` , and `Settlement` .

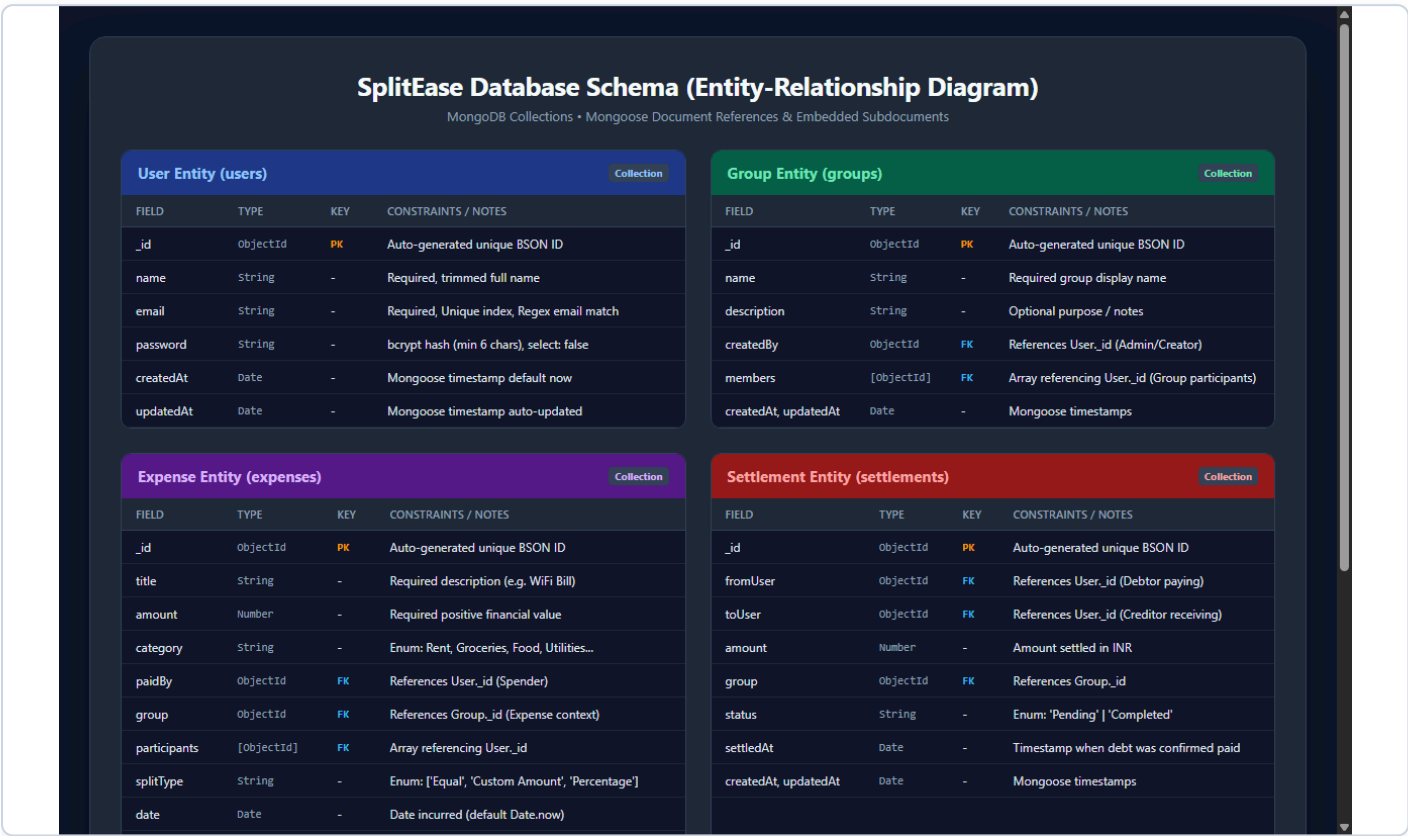


Figure 4.1: Database Schema & Entity Relationships (Rendered from Draw.io source)

4.2 Subdocument Embedding vs. Referential Modeling

A critical architectural decision was embedding `splits: [splitSchema]` directly within the `Expense` document rather than maintaining a separate relational table:

- Atomic Reads & Writes:** When an expense is recorded or fetched, its split allocations are read in a single $O(1)$ document lookup, avoiding expensive multi-table joins.
- Schema Flexibility:** Supports Equal, Custom Amount, and Percentage splits dynamically without altering relational column definitions.

5. Core Algorithms & Mathematical Formulations

5.1 Greedy Two-Pointer Debt Simplification Algorithm

In a shared flat with (N) members, naive peer-to-peer settling requires up to $\frac{N(N-1)}{2}$ transfers $(O(N^2))$. SplitEase applies a **Greedy Two-Pointer Optimization Algorithm** that reduces total transactions to at most $(N-1)$.

1. Compute net balance $(B(u))$ for each user (u) : $B(u) = \text{ext}\{\text{Total Paid}\} - \text{ext}\{\text{Total Owed}\} + \text{ext}\{\text{Settlements Received/Paid}\}$.
2. Partition members into two disjoint sets: **Debtors** $(B < 0)$ and **Creditors** $(B > 0)$.
3. Sort both sets descending by absolute balance amount.
4. Iteratively match the largest debtor with the largest creditor, settling $\min(|B_d|, B_c)$, updating remaining dues until both lists are exhausted.

Code Snippet 5: Greedy Debt Simplification Engine (server/src/controllers/balanceController.js)

```
// Greedy Two-Pointer Debt Simplification - O(N log N)
const debtors = [];
const creditors = [];

Object.keys(rawBalances).forEach(userId => {
  const balance = Number(rawBalances[userId].toFixed(2));
  if (balance < 0) debtors.push({ userId, amount: -balance });
  else if (balance > 0) creditors.push({ userId, amount: balance });
});

// Sort descending to maximize transaction settlement per step
debtors.sort((a, b) => b.amount - a.amount);
creditors.sort((a, b) => b.amount - a.amount);

let transactions = [];
let i = 0, j = 0;

while (i < debtors.length && j < creditors.length) {
  const debtor = debtors[i];
  const creditor = creditors[j];
  const amountToSettle = Math.min(debtor.amount, creditor.amount);

  transactions.push({
    from: userMap[debtor.userId],
    to: userMap[creditor.userId],
    amount: Number(amountToSettle.toFixed(2))
  });

  debtor.amount -= amountToSettle;
  creditor.amount -= amountToSettle;

  if (Math.abs(debtor.amount) < 0.01) i++;
  if (Math.abs(creditor.amount) < 0.01) j++;
}
```

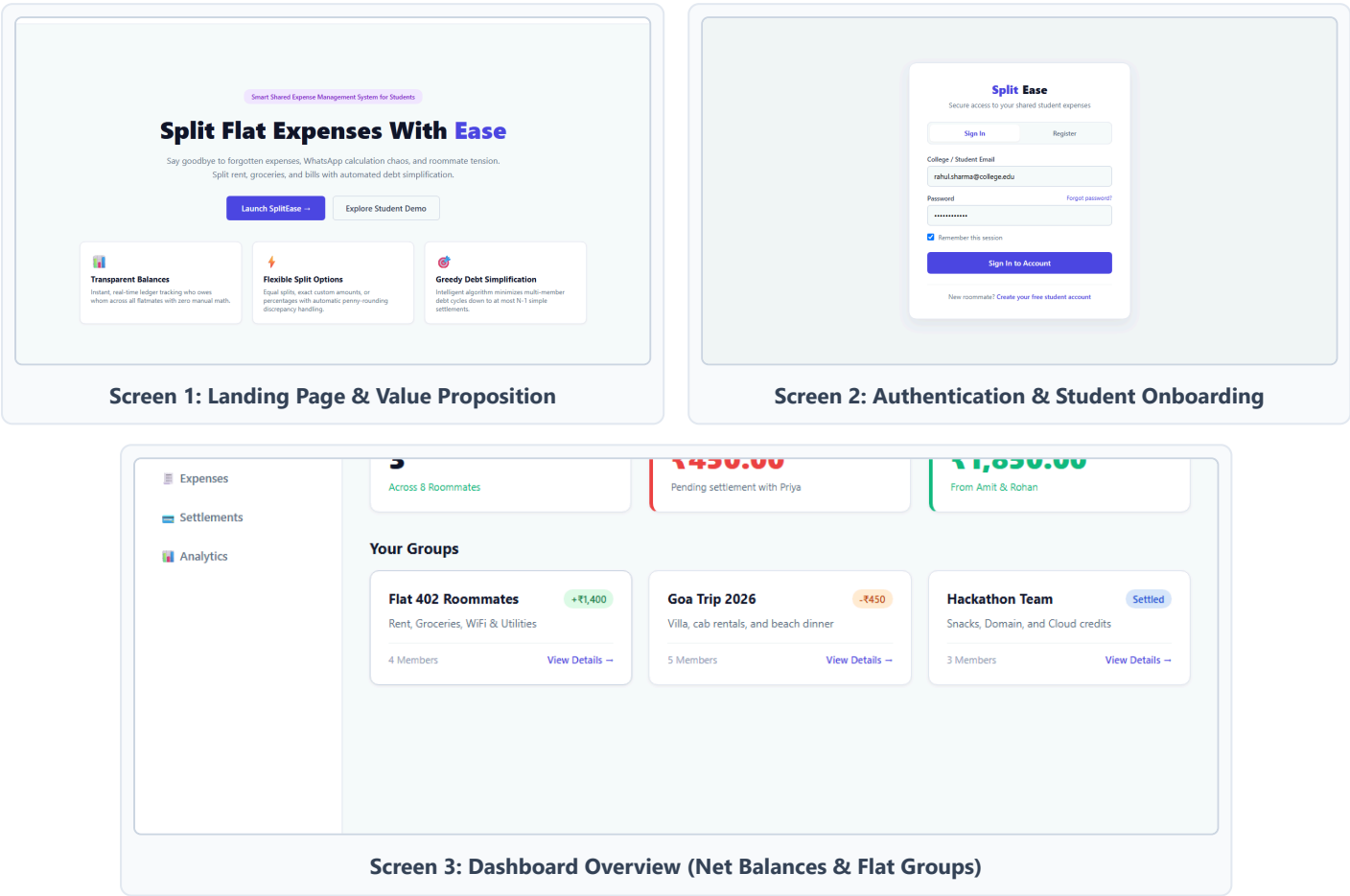
5.2 Penny-Rounding Discrepancy Allocation

When dividing bills that produce recurring decimals (e.g. ₹100 / 3 = 33.3333...), naive rounding produces an unbalanced split: $(33.33 \times 3 = 99.99)$ (₹0.01 lost). SplitEase's `expenseController.js` calculates $(\Delta = \text{ext}\{\text{amount}\} - \sum \text{ext}\{\text{splitAmount}\})$ and automatically allocates (Δ) into the primary participant, ensuring 100% mathematical closure.

6. User Interface Design & Usability Engineering

6.1 Figma Prototype & Design System

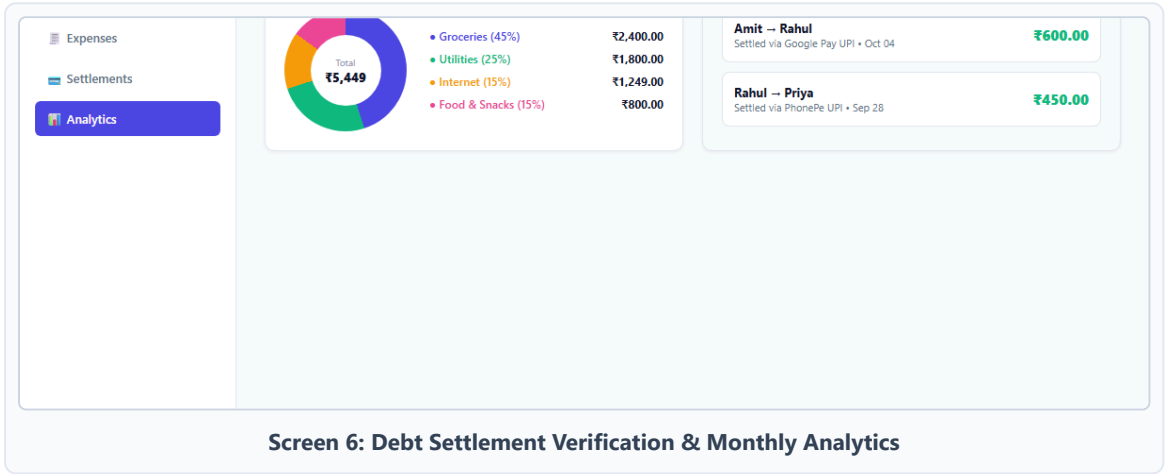
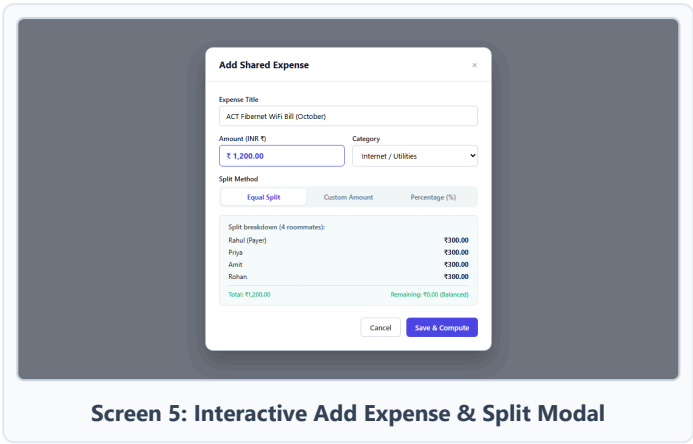
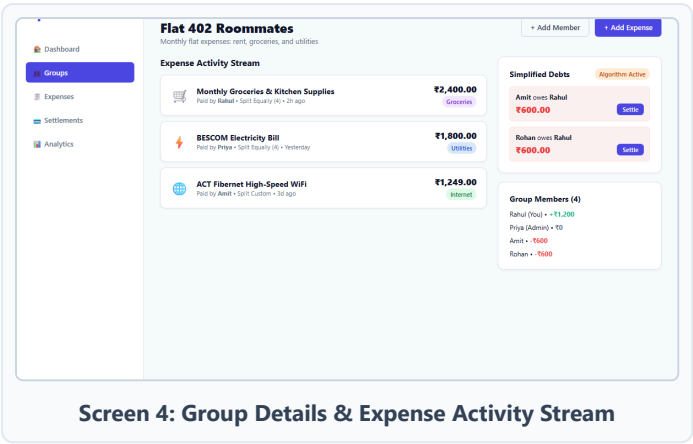
The user interface was prototyped in **Figma** adhering to modern student-friendly design heuristics: minimal cognitive load, high balance contrast, 8pt spatial grid, and instant visual validation. **Interactive Prototype:** [SplitEase Figma Interactive Prototype](#).



6.2 Usability Engineering Highlights (Screens 1 to 3)

- **5-Second Value Clarity (Screen 1):** Prominent CTAs allow student roommates to test or register immediately without encountering marketing fluff.
- **Frictionless 10-Second Sign Up (Screen 2):** Only requires student email, name, and password with automatic JWT session persistence.
- **Immediate Cognitive Answer (Screen 3):** High-contrast summary cards instantly display "You Owe" (Semantic Red) and "You Are Owed" (Semantic Green).

6. User Interface Design (Continued: Screens 4 to 6)



6.3 Usability Engineering Highlights (Screens 4 to 6)

- **Single Source of Truth (Screen 4):** Dual-column layout pairs a real-time chronological expense feed with live simplified debts and member avatars.
- **Proactive Error Prevention (Screen 5):** Split calculation tabs validate that allocations equal the expense amount, displaying real-time balance pills before submission.
- **Frictionless Settlement Flow (Screen 6):** Roommates mark debts settled with a single click, recording UPI payment confirmations and instantly zeroing out ledger balances.

7. Key Design Decisions & Trade-Off Analysis

During the engineering lifecycle of SplitEase, five foundational technical decisions were evaluated against trade-offs to optimize for student flatmate ergonomics:

Decision	Chosen Approach	Why & Trade-Off Justification
1. Debt Simplification Algorithm	Greedy Two-Pointer matching ($O(N \log N)$)	Why: Minimizes total payment transactions from $O(N^2)$ to at most $(N-1)$. Eliminates roommate confusion and circular debt transfers.
2. Persistence Paradigm	Document-Oriented NoSQL (MongoDB + Mongoose)	Why: Expense splits naturally represent embedded documents. Allows flexible Equal, Custom, and Percentage splits without complex SQL table joins.
3. Authentication Mechanism	Stateless JWT with Client LocalStorage & Interceptors	Why: Eliminates server session state, simplifies containerized multi-instance deployment, and provides instantaneous route-guarding.
4. Off-Chain Manual Settlement	Manual "Mark Settled" audit records over Gateway	Why: Students universally settle via Google Pay/PhonePe/cash. Avoids banking KYC friction, payment processing charges, and gateway maintenance.
5. Currency & Math Normalization	Explicit INR (₹) formatting + Penny Discrepancy Allocation	Why: Protects financial consistency across division of recurring fractional numbers (e.g. ₹100 / 3) by absorbing rounding diffs into payer.

7.1 Verification Strategy & Future Roadmap

- Automated Testing:** Unit tests for split apportionment, penny rounding distribution, and two-pointer creditor/debtor matching.
- Container Verification:** Continuous verification via `docker-compose up --build` ensuring isolated reproducibility.
- Future Roadmap:** Optical Character Recognition (OCR) receipt scanning via Tesseract, push notification payment reminders, and multi-flat portfolio analytics.

SplitEase Architecture Approved • Designed for Students, Built with Engineering Rigor.